

COMP3242/COMP6242 – Deep Learning: Practice Exam

Australian National University

Examiner: Stephen Gould

June 2026

INSTRUCTIONS:

1. This is a three hour written examination with 15 minutes reading time. You are allowed to use text books, hand written notes, and printed notes. Electronic devices are **not** allowed.
2. The exam consists of five sections of equal value. Each section contains a number of questions. Marks allocated for each question are shown next to the question.
3. Students enrolled in COMP6242 are required to answer all questions. Students enrolled in COMP3242 can choose any four of the five sections to answer.
4. Start each section on a new page.
5. Show all working.

Good luck.

Solution Sketch. Example answers are provided. These are not necessarily complete and we would expect more explanation or more detailed steps that show your full working in the exam.

Section A.

Answer each of the following questions [marks per questions as shown; 20 total]

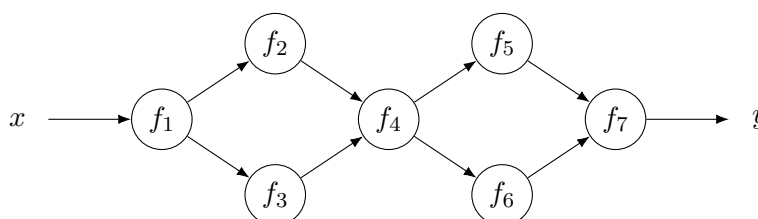
1. **Partial Derivatives (5 marks).** Given the function $f(x; a, b) = a^\top x + b$ for $a, x \in \mathbb{R}^n$ and $b \in \mathbb{R}$, compute all partial derivatives,

$$\frac{\partial f}{\partial x}, \quad \frac{\partial f}{\partial a}, \quad \frac{\partial f}{\partial b}.$$

Solution Sketch.

$$\frac{\partial f}{\partial x} = a, \quad \frac{\partial f}{\partial a} = x, \quad \frac{\partial f}{\partial b} = 1.$$

2. **Data Flow Graphs (7 marks).** Consider the following data flow graph. You can assume that $x \in \mathbb{R}^n$ and $y \in \mathbb{R}$.



- (a) Write down the function $x \mapsto y$ corresponding to the graph. You may introduce intermediate variables if it helps simplify your answer.
- (b) Write down an expression for the gradient of the output of the graph y with respect to the input of the graph x . Show all working.

Solution Sketch. Let $z = f_4(f_2(f_1(x)), f_3(f_1(x)))$. Then $y = f_7(f_5(z), f_6(z))$.

$$\frac{dy}{dx} = \left(\frac{\partial f_7}{\partial f_5} \frac{\partial f_5}{\partial f_4} + \frac{\partial f_7}{\partial f_6} \frac{\partial f_6}{\partial f_4} \right) \left(\frac{\partial f_4}{\partial f_2} \frac{\partial f_2}{\partial f_1} + \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_1} \right) \frac{\partial f_1}{\partial x}$$

3. **Multi-class Logistic (8 marks).** Consider the multi-class logistic function

$$P(y = k \mid x) = \frac{\exp(a_k^\top x + b_k)}{\sum_{j=1}^K \exp(a_j^\top x + b_j)}$$

with parameters $(a_k, b_k) \in \mathbb{R}^n \times \mathbb{R}$ for $k = 1, \dots, K$. Show that the function is over-parametrized. That is, show that there exists a different set of parameters $(c_k, d_k) \in \mathbb{R}^n \times \mathbb{R}$ for $k = 1, \dots, K$ such that

$$\frac{\exp(a_k^\top x + b_k)}{\sum_{j=1}^K \exp(a_j^\top x + b_j)} = \frac{\exp(c_k^\top x + d_k)}{\sum_{j=1}^K \exp(c_j^\top x + d_j)}$$

for all $x \in \mathbb{R}^n$.

Solution Sketch. Let $c_k = a_k - a_K$ and $b_k = b_k - b_K$ for $k = 1, \dots, K$. Then the result is satisfied since we can multiply the LHS by the identity

$$\frac{\exp(a_K^\top x + b_K)}{\exp(a_K^\top x + b_K)}$$

and factor without changing the value of the expression.

Section B.

Answer each of the following questions [marks per questions as shown; 20 total]

1. **1D Deconvolution (5 marks).** Consider input signal $x = (6, 1, 2, 5, 8, 6, 4, 2)$. Let $a = (-1, 1, 2)$ be a filter kernel. Compute the output signal y obtained from deconvolving x with a using a stride of 2.

Solution Sketch. $y = (-6, 6, 11, 1, 0, 2, -1, 5, 2, 8, 10, 6, 8, 4, 6, 2, 4)$

2. **Upsampling (8 marks).** Consider again the signal $x = (6, 1, 2, 5, 8, 6, 4, 2)$. We wish to upsample the signal to three times its original length. Determine the upsampled signal using the following methods.

- (a) Nearest neighbour.
- (b) Linear interpolation.

Show working and state any assumptions.

Solution Sketch.

- (a) $y = (6, 6, 6, 1, 1, 1, 2, 2, 2, 5, 5, 5, 8, 8, 8, 6, 6, 6, 4, 4, 4, 2, 2, 2)$
- (b) $y = (4, 6, 4.3, 2.7, 1, 1.3, 1.7, 2, 3, 4, 5, 6, 7, 8, 7.3, 6.7, 6, 5.3, 4.7, 4, 3.3, 2.7, 2, 1.3)$ where we have assumed zero padding at the edges

3. **Recurrent Neural Network (7 marks).** Consider the linear dynamical system

$$\begin{aligned}y_t &= Ax_t + Bh_{t-1} + c \\h_t &= Dx_t + Eh_{t-1} + f\end{aligned}$$

with $h_0 = 0$. Here $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times h}$, $c \in \mathbb{R}^n$, $D \in \mathbb{R}^{h \times n}$, $E \in \mathbb{R}^{h \times h}$, and $f \in \mathbb{R}^h$ are the model parameters.

- (a) Argue that this is a recurrent neural network (albeit a very simple one).
- (b) Show that we can write y_t as a linear function of $x_1, \dots, x_t$.

Solution Sketch.

- (a) The output at time t depends on the input at time t and a hidden state at time t , which is updated recurrently based on the current input previous hidden state.
- (b) Write h_t as

$$\begin{aligned}h_t &= Dx_t + E(Dx_{t-1} + E(\dots) + f) + f \\&= \sum_{k=0}^{t-1} E^k Dx_{t-k} + \sum_{k=0}^{t-1} E^k f\end{aligned}$$

Then

$$\begin{aligned}
y_t &= Ax_t + B \left(\sum_{k=0}^{t-2} E^k D x_{t-k-1} + \sum_{k=0}^{t-2} E^k f \right) + c \\
&= \begin{bmatrix} BE^{t-2}D & 0 & 0 & \dots & 0 & 0 \\ 0 & BE^{t-3}D & 0 & \dots & 0 & 0 \\ 0 & 0 & BE^{t-4}D & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & BD & 0 \\ 0 & 0 & 0 & \dots & 0 & A \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{t-1} \\ x_t \end{bmatrix} + B \sum_{k=0}^{t-2} E^k f + c
\end{aligned}$$

Section C.

Answer each of the following questions [marks per questions as shown; 20 total]

1. **Online Softmax (10 marks).** Consider the numerically stable expression for softmax,

$$\text{softmax}(z) = \left(\frac{\exp(z_1 - z_{\max})}{Z}, \dots, \frac{\exp(z_n - z_{\max})}{Z} \right)$$

where $z_{\max} = \max\{z_1, \dots, z_n\}$ and $Z = \sum_{i=1}^n \exp(z_i - z_{\max})$. The online softmax algorithm calculates $z_{\max}$ and Z in one pass as

```
1: function ONLINESOFTMAX( $z$ )
2:    $z_{\max}, Z = -\infty, 0$                                 ▷ initialize
3:   for all  $i = 1, \dots, n$  do
4:     if  $z_i > z_{\max}$  then                                ▷ new max element
5:        $Z \leftarrow Z \exp(z_{\max} - z_i)$                   ▷ re-scale  $Z$ 
6:        $z_{\max} \leftarrow z_i$                              ▷ update new maximum
7:     end if
8:      $Z \leftarrow Z + \exp(z_i - z_{\max})$                 ▷ accumulate  $Z$ 
9:   end for
10:  return  $z_{\max}, Z$ 
11: end function
```

Prove that the online softmax algorithm is correct. That is, show that the code correctly implements the equation for softmax. You may assume infinite precision arithmetic.

Solution Sketch. Consider the i -th iterate. We have from Lines 4 and 6 that $z_{\max} = \max\{z_1, \dots, z_i\}$. So by the last iterate, $i = n$, we have the correct $z_{\max}$. Now, whenever we find a new $z_{\max}$ we rescale the previous partial sum Z by $\exp(z_{\text{old max}} - z_{\text{new max}})$ on Line 5, which changes every term in the partial sum from $\exp(z_i - z_{\text{old max}})$ to $\exp(z_i - z_{\text{new max}})$. The final sum is therefore correct.

More formally, define $z_{\max}^{(k)} = \max\{z_1, \dots, z_k\}$ and $Z^{(k)} = \sum_{i=1}^k \exp(z_i - z_{\max}^{(k)})$. We now prove correctness of the code by induction. In particular we show that at the k -th iteration variable $z_{\max} = z_{\max}^{(k)}$ and variable $Z = Z^{(k)}$.

Base Case: At iteration $k = 1$ we execute Lines 5, 6 and 7. Line 5 results in $Z = 0$, Line 6 results in $z_{\max} = z_1 = z_{\max}^{(1)}$ and Line 7 results in $Z = e^0 = \exp(z_1 - z_{\max}^{(1)})$.

Inductive Step: Assume true for $k = i$. Consider $k = i + 1$. If $z_{i+1} > z_{\max}^{(i)}$ then $z_{\max}^{(i+1)} = z_{i+1}$

which is assigned to $z_{\max}$ on Line 6, and

$$\begin{aligned}
 Z &= \underbrace{Z^{(i)} \exp(z_{\max}^{(i)} - z_{i+1}) + \exp(z_{i+1} - z_{\max}^{(i+1)})}_{\text{Line 5}} \\
 &\quad \underbrace{\hspace{10em}}_{\text{Line 8}} \\
 &= \sum_{j=1}^i \exp(z_j - z_{\max}^{(i)}) \exp(z_{\max}^{(i)} - z_{i+1}) + \exp(z_{i+1} - z_{\max}^{(i+1)}) \\
 &= \sum_{j=1}^i \exp(z_j - z_{i+1}) + \exp(z_{i+1} - z_{\max}^{(i+1)}) \\
 &= \sum_{j=1}^{i+1} \exp(z_j - z_{\max}^{(i+1)}) \\
 &= Z^{(i+1)}
 \end{aligned}$$

Otherwise $z_{\max}^{(i+1)} = z_{\max}^{(i)}$ and variable $z_{\max}$ is unchanged. And from Line 8

$$Z = Z^{(i)} + \exp(z_{i+1} - z_{\max}^{(i+1)}) = Z^{(i+1)}$$

Therefore if true for $k = i$ also true for $k = i + 1$.

Conclusion: Setting $k = n$ proves correctness of the code.

2. **Transformers (10 marks).** Recall the scaled dot-product attention equation for single query q during inference is,

$$z = \mathbf{softmax} \left(\frac{q^\top K^\top}{\sqrt{d}} \right) V$$

Given

$$q = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix}, \quad V = \begin{bmatrix} 7 & 1 & 3 \\ 4 & 2 & 6 \end{bmatrix}$$

Calculate z . You can leave your answer in terms of e and $\sqrt{\cdot}$. Show all working.

Solution Sketch.

$$\begin{aligned}
 \frac{q^\top K^\top}{\sqrt{d}} &= \frac{1}{\sqrt{3}} \begin{bmatrix} 2 & 0 \end{bmatrix} \\
 \mathbf{softmax} \left(\frac{1}{\sqrt{3}} \begin{bmatrix} 2 & 0 \end{bmatrix} \right) &= \frac{1}{\exp\left(\frac{2}{\sqrt{3}}\right) + 1} \begin{bmatrix} \exp\left(\frac{2}{\sqrt{3}}\right) & 1 \end{bmatrix} \\
 z &= \frac{1}{\exp\left(\frac{2}{\sqrt{3}}\right) + 1} \begin{bmatrix} 7 \exp\left(\frac{2}{\sqrt{3}}\right) + 4 & \exp\left(\frac{2}{\sqrt{3}}\right) + 2 & 3 \exp\left(\frac{2}{\sqrt{3}}\right) + 6 \end{bmatrix}
 \end{aligned}$$

Section D.

Answer each of the following questions [marks per questions as shown; 20 total]

In this question we will derive some theoretical results.

1. **ELBO (10 marks).** Consider variational inference where the goal is to maximize the marginal log-likelihood of the observed data

$$\log p(x) = \log \left(\int_z p(x, z) dz \right)$$

- (a) Introduce conditional distribution $q(z | x)$ explain why

$$\int_z p(x, z) dz = \int_z q(z | x) \frac{p(x, z)}{q(z | x)} dz$$

- (b) Jensen's inequality states that $\log \int \psi(z) f(z) dz \geq \int \psi(z) \log f(z) dz$ for any distribution ψ and function f . Use this to show that

$$\begin{aligned} \log \left(\int_z q(z | x) \frac{p(x, z)}{q(z | x)} dz \right) &\geq \int_z q(z | x) \log \left(\frac{p(x, z)}{q(z | x)} \right) dz \\ &= E_{q(z|x)} \left[\log \frac{p(x, z)}{q(z | x)} \right] \end{aligned}$$

where $E[\cdot]$ computes the expected value of its argument.

- (c) Factor $p(x, z)$ as $p(x | z)p(z)$ and show how to obtain the evidence lower bound

$$\log p(x) \geq E_{q(z|x)} [\log p(x | z)] - E_{q(z|x)} \left[\log \frac{q(z | x)}{p(z)} \right]$$

Solution Sketch.

- (a) $q(z | x)/q(z | x) = 1$
(b) Line 1: let $\psi(z) = p(z | x)$ and $f(z) = \frac{p(x, z)}{q(z | x)}$, and apply Jensen's inequality. Line 2: Rewrite the integral as an expected value.
(c) Write

$$\log \frac{p(x, z)}{q(z | x)} = \log \frac{p(x | z)p(z)}{q(z | x)} = \log p(x | z) - \log \frac{q(z | x)}{p(z)}$$

and distribute the expectation (which is linear)

2. **DDIM (10 marks).** Denoising diffusion implicit models (DDIM) is a variant of the denoising diffusion probabilistic model (DDPM) shown in class that uses deterministic sampling. DDIM starts by deriving an alternative update equation for the reverse diffusion process as

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\epsilon_\theta(x_t, t) + \sigma_t z$$

where $z \sim \mathcal{N}(0, I)$ is zero-mean unit-variance noise, σ_t controls the amount of noise injected in the reverse step, and $\epsilon_\theta(x_t, t)$ is the noise predicted by the network.

- (a) By setting σ_t to zero, show that we end up with a deterministic update for x_{t-1} .

- (b) We will now argue that this is analogous to solving an ordinary differential equation (ODE),

$$\frac{dx}{dt} = -\frac{1}{2}\beta(t)x + \frac{1}{2}\beta(t)\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ and β_t is assumed to be small. *Hint.* An ODE, $\frac{dy}{dt} = f(y, t)$, can be approximately solved using Euler's method (with step size $h = 1$) as

$$y_{t+1} = y_t + f(t, y_t).$$

- (i) Using $x_0 \approx \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$ show that

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}} + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)$$

- (ii) Using the approximations

$$\begin{aligned} \sqrt{\frac{\bar{\alpha}_{t-1}}{\bar{\alpha}_t}} &= \frac{1}{\sqrt{\bar{\alpha}_t}} \approx 1 + \frac{\beta_t}{2} \\ \sqrt{1-\bar{\alpha}_{t-1}} - \sqrt{\frac{\bar{\alpha}_{t-1}}{\bar{\alpha}_t}}\sqrt{1-\bar{\alpha}_t} &\approx -\frac{\beta_t}{2\sqrt{1-\bar{\alpha}_t}} \end{aligned}$$

for small β_t , rewrite the expression for x_{t-1} .

- (iii) Relate this to Euler's method for approximating solving an ODE.

Solution Sketch.

- (a) Setting $\sigma_t = 0$ we get

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon_\theta(x_t, t)$$

which is a deterministic function of x_t and t .

- (b) (i) follows from direct substitution. (ii) gives

$$\begin{aligned} x_{t-1} &= \left(1 - \frac{1}{2}\beta_t\right)x_t + \frac{1}{2}\beta_t\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \\ &= x_t + f(t, x_t) \end{aligned}$$

where $f(t, x_t) = -\frac{1}{2}\beta_t x_t + \frac{1}{2}\beta_t \frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}$ from which (iii) follows by reversing time.

Section E.

Answer each of the following questions [marks per questions as shown; 20 total]

Consider the following snippet of PyTorch code and answer the questions below.

```
1 import torch
2 import torch.nn as nn
3
4 class MyMLP(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.f = nn.Sequential(
8             nn.Linear(4, 8, bias=True),
9             nn.ReLU(),
10            nn.Linear(8, 2, bias=True)
11        )
12
13    def forward(self, x):
14        return self.f(x)
15
16 model = MyMLP()
17
18 x = torch.tensor([[1.0, 0.5, -1.0, 2.0]])
19 target = torch.tensor([1])
20
21 criterion = nn.CrossEntropyLoss()
22 optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
23
24 for iteration in range(10):
25     output = model(x)
26     loss = criterion(output, target)
27
28     optimizer.zero_grad()
29     loss.backward()
30     optimizer.step()
```

1. (2 marks). What is the shape of `output`?
2. (4 marks). How many learnable parameters are there in the model?
3. (4 marks). What does `loss.backward()` on Line 29 compute?
4. (4 marks). What happens if `optimizer.zero_grad()` on Line 28 is omitted? What happens if `optimizer.step()` on Line 30 is omitted?
5. (4 marks). How many gradient steps are taken? Can we increase the learning rate and take only one gradient step to get to the same final parameters?
6. (2 marks). What do you expect the final value of `output` to be?

Solution Sketch.

1. (1, 2)
2. $(4 \times 8 + 1) + (8 \times 2 + 1) = 50$

3. The gradient of the parameters with respect to the `loss` variable.
 4. If `optimizer.zero_grad()` is omitted then the gradient is not zeroed and therefore accumulates over iterations. If `optimizer.step()` is omitted then the gradient update is never applied and the parameters never change.
 5. Ten gradient steps are taken. Increasing the learning rate will make the steps bigger but since this is not a linear objective the parameters won't converge to the same values.
 6. We expect `output` to approach $(-\infty, \infty)$, but more likely $(-\alpha, \alpha)$ for some small α after the small number of steps.
-